UNIT 5 · TOPIC 5.1

# Application Code Vulnerabilities

Assignment

*Instructions: 15 practice questions + 4 coding assignment(s), 35 points total. Show your work for short-answer items. Coding assignments may be completed here or in the interactive auto-graded environment on the course site (same problem, same points).*

**Name:** **Date:** **Class:**

## Practice Questions

### 5.1.a · SQL injection

**1. Which of the following describes how a SQL injection attack is executed?** (1 pt)

- ○ A. An attacker floods the database server with connections until it crashes
- ○ B. An attacker intercepts database traffic using a packet sniffer
- ○ C. An attacker inputs SQL commands into a user field, which the database then parses and runs as executable code
- ○ D. An attacker changes the routing tables to redirect database requests to their own host

**2. Which of the following methods are effective practices to prevent or mitigate SQL injection attacks? (Select ALL that apply.)** (3 pts)

- □ A. Using parameterized queries (prepared statements)
- □ B. Encrypting database tables using AES symmetric encryption
- □ C. Using input validation and sanitization
- □ D. Running the database application with least-privilege permissions

*(Select ALL that apply.)*

**3. What is the name of the SQL injection technique that uses a database keyword to combine results from the original query with results from a table of the attacker's choosing?** (1 pt)

### 5.1.b · Cross-site scripting (XSS)

**4. A search page prints 'You searched for: <user input>' straight into the results HTML, with no escaping. Which vulnerability class is this?** (1 pt)

- ○ A. SQL injection
- ○ B. Broken object-level authorization
- ○ C. Cross-site scripting
- ○ D. Denial of service

**5. Select ALL statements that correctly describe stored XSS.** (3 pts)

- □ A. The payload is saved server-side (e.g., in a database or comment field)
- □ B. Every user who later views the infected page can be compromised, with no extra click needed

□ C. It requires the victim to click a specially crafted URL each time

□ D. It is generally considered more dangerous than reflected XSS because no social engineering step is needed

*(Select ALL that apply.)*

**6. Why is output encoding (escaping) considered the primary defense against XSS, rather than input validation alone?** (1 pt)

○ A. Input validation cannot block angle brackets without breaking normal text

○ B. Escaping also prevents SQL injection, so one control covers both

○ C. The same string can be safe in one context (plain text) and dangerous in another (rendered as HTML), so escaping must happen at the point of output

○ D. Escaping runs in the browser, so the server does not need to change

**7. Name the XSS variant where the vulnerability lives entirely in client-side JavaScript and never touches the server.** (1 pt)

### 5.1.c · API vulnerabilities

**8. A logged-in user changes the URL from /api/accounts/482 to /api/accounts/483 and the API returns account 483's private balance. What is this flaw called?** (1 pt)

○ A. SQL injection

○ B. Denial of service

○ C. Cross-site scripting

○ D. Broken Object Level Authorization (BOLA/IDOR)

**9. Select ALL statements that correctly distinguish authentication from authorization in the context of BOLA.** (3 pts)

□ A. Authentication proves who the caller is (e.g., a valid session token)

□ B. Authorization checks whether that identity is allowed to access a specific object

□ C. BOLA is caused by an authentication failure, not an authorization failure

□ D. A valid login token alone does not prove the caller owns every object they might request

*(Select ALL that apply.)*

**10. Why does hiding a button in the app's user interface fail to prevent a BOLA attack?** (1 pt)

○ A. Hidden buttons still appear in the page source, so users can click them

○ B. Attackers can call the API endpoint directly with tools like curl or Postman, bypassing the UI entirely

○ C. The API caches the hidden button's response for other users

○ D. Hiding the button only works on mobile apps, not websites

**11. Name the API flaw where an endpoint returns a full internal object (extra fields like password hashes) and relies on the client to hide sensitive fields.** (1 pt)

### 5.1.d · Data breaches and their impact

**12. Which factor is NOT one of the three primary drivers of a data breach's severity discussed in this atom?** (1 pt)

- ○ A. Sensitivity of the data exposed
- ○ B. Scale (number of records)
- ○ C. Whether the attacker was an insider or an outsider
- ○ D. Time to detect and contain

**13. Select ALL data categories that count as high-sensitivity (PII/PHI/financial/authentication secrets).** (3 pts)

- □ A. Full name plus Social Security number
- □ B. Public blog post titles
- □ C. Plaintext passwords
- □ D. Credit card numbers

*(Select ALL that apply.)*

**14. Why is a breach of hashed-and-salted passwords generally considered less severe than a breach of the same passwords in plaintext?** (1 pt)

- ○ A. Hashed passwords are encrypted, so attackers would need the server's key
- ○ B. Hashing adds a layer of work an attacker must still defeat before the passwords are usable
- ○ C. Salted hashes can never be cracked, so the breach is harmless
- ○ D. Users with hashed passwords do not need to change them after a breach

**15. Name the general incident-response step that comes immediately after containment: identifying what data and how many records/individuals were affected.** (1 pt)

## Coding Assignments (Python 3)

### 5.1.a · SQL injection

*3 pts · auto-graded on the site*

#### Coding problem 5.1.a: Fix the Injectable Login

**Skill:** Mitigate Risk · **Maps to:** 5.1.A, 5.1.B · **Language:** Python 3 (pytest, sqlite3)

#### The situation

The `authenticate` function below is the login check for a small app. It works for normal users, but it builds its SQL query by **concatenating** the username and password straight into the query text. That is the exact flaw that makes SQL injection possible: an attacker who types SQL syntax instead of a username can change what the query *means*.

The starter code is the vulnerable version. Run the tests and you will see the injection cases fail: inputs like `admin'--` and `' OR '1'='1` log in **without the real password**.

#### Your task

Rewrite `authenticate` so that user input can never change the structure of the query. Use a **parameterized query** (a prepared statement): pass the SQL template with `?` placeholders and the user values as a separate tuple, and let the database driver bind them as data.

```
cur.execute("SELECT role FROM users WHERE username = ? AND password = ?", (username, password))
```

When you do this, the injection inputs are treated as ordinary (non-existent) usernames, so they return no row, while genuine credentials still succeed.

### Function contract

Implement `authenticate(users, username, password) -> str | None` in `starter.py`:

- `users` is a list of `(username, role, password)` tuples to load into an in-memory table.
- Return the matching user's `role` if (and only if) both username and password match a row.
- Return None for any failed or injected login.

Do **not** filter or blocklist the input, and do **not** compare the lists in Python. The whole point is that the *database query itself* must be safe. Use `sqlite3` with `:memory:`.

### How to run

```
cd problem && python -m pytest tests.py -q
```

A correct solution passes every test, including the injection cases. A string-concatenation version will fail them, because the injected input logs in.

### Starter code:

```
import sqlite3


def authenticate(users, username, password):
    """
    Return the role of the user whose username AND password both match a row,
    or None if the login fails.

    VULNERABLE starter: it concatenates user input into the SQL text, so an
    attacker can inject SQL. Rewrite the query to use parameter placeholders (?)
    with the values passed separately, so input can never change the query's
    structure. Do not blocklist input; fix the query itself.
    """
    conn = sqlite3.connect(":memory:")
    cur = conn.cursor()
    cur.execute("CREATE TABLE users (username TEXT, role TEXT, password TEXT)")
    cur.executemany("INSERT INTO users VALUES (?, ?, ?)", users)

    # --- the injectable part: user input is glued straight into the SQL text ---
    query = (
        "SELECT role FROM users WHERE username = '"
        + username + "' AND password = '" + password + "'"
    )
    try:
        cur.execute(query)
        row = cur.fetchone()
    except sqlite3.Error:
        row = None
    conn.close()
    return row[0] if row else None
```

## 5.1.b · Cross-site scripting (XSS)

*3 pts · auto-graded on the site*

## Coding problem 5.1.b: Fix the Reflected Comment

**Skill:** Mitigate Risk · **Maps to:** 5.1.A, 5.1.B · **Language:** Python 3 (pytest)

### The situation

`render_comment` builds the HTML that shows a visitor's comment on a page. It works for ordinary text, but it drops the username and comment **straight into the HTML** by string concatenation. That is reflected cross-site scripting (XSS): whatever the user typed is parsed by the next visitor's browser as markup, so `<script>...</script>` in a comment becomes a live script running in that visitor's session.

The starter is the vulnerable version. A comment of `<script>steal()</script>` comes back out as a real `<script>` tag.

### Your task

Fix `render_comment` so user input can never be parsed as HTML. The correct defense is **context-aware output encoding**: HTML-encode every user-supplied value before it goes into the page, so < becomes `<`, > becomes `>`, & becomes `&`, and the quote characters are encoded too. Python's standard library does exactly this:

```
import html
safe = html.escape(user_value, quote=True)
```

Encode the `username` and `comment` values, then place them into the same HTML template. Do **not** detect, strip, or blocklist payloads — the point is that *encoded* input is harmless no matter what it contains, while the app's own structural tags stay intact.

### Function contract

Implement `render_comment(username: str, comment: str) -> str`:

- Return the comment card HTML: `<div class="c"><b>USERNAME</b>: COMMENT</div>`, where `USERNAME` and `COMMENT` are the HTML-encoded user values.
- The structural tags (`<div class="c">`, `<b>`) are the app's own output and stay as real HTML.

### How to run

```
cd problem && python -m pytest tests.py -q
```

A correct solution passes every test, including the injection cases. The concatenation version fails them because the raw `<script>` and `<img onerror=...>` payloads reach the output as markup.

### Starter code:

```
import html  # noqa: F401  (you will need this to fix the function)


def render_comment(username: str, comment: str) -> str:
    """
    Return the HTML card that displays a visitor's comment:

        <div class="c"><b>USERNAME</b>: COMMENT</div>

    VULNERABLE starter: it concatenates the username and comment straight into
    the HTML, so a comment like <script>steal()</script> is sent to the next
    visitor's browser as a real script tag (reflected XSS).

    Fix it with context-aware OUTPUT ENCODING: HTML-encode each user value
    (html.escape(value, quote=True)) before placing it into the template, so the
    browser can never parse user input as markup. Do not strip or blocklist input;
    keep the structural <div>/<b> tags as real HTML.
```

```
    """
    return '<div class="c"><b>' + username + "</b>: " + comment + "</div>"
```

### 5.1.c · API vulnerabilities

*3 pts · auto-graded on the site*

## Coding problem 5.1.c: Detect broken object-level authorization (BOLA)

**Skill:** Analyze Risk · **Maps to:** 5.1.B · **Language:** Python 3 (pytest)

### Problem

Build the server-side authorization check an API gateway should run before returning an object. Given the authenticated caller's user id, the id of the object they are requesting, an object-ownership map, and the caller's role, decide whether the request is allowed.

Rules:

- A request is **allowed** if the caller owns the requested object (the object's `owner` equals the caller's `user_id`), OR the caller's `role` is `"admin"`.
- A request is **denied** (a BOLA violation) if the object exists but the caller neither owns it nor is an admin.
- A request for an object id that does not exist in the ownership map is **denied** with reason `"not_found"`.

### Function contract

Implement `solve(request: dict, ownership: dict) -> dict` in `starter.py` (see `interface.md` / `blackbox.yaml`).

### Input

- `request`: `{"user_id": str, "role": str, "object_id": str}`
- `ownership`: `dict` mapping `object_id -> owner user_id` (e.g. `{"482": "u1", "483": "u2"}`)

### Output

A dict: `{"allowed": bool, "reason": str}` where `reason` is one of `"owner"`, `"admin"`, `"not_owner"`, `"not_found"`.

### Examples

```
request={"user_id":"u1","role":"user","object_id":"482"}, ownership={"482":"u1"}
    -> {"allowed": True, "reason": "owner"}

request={"user_id":"u1","role":"user","object_id":"483"}, ownership={"483":"u2"}
    -> {"allowed": False, "reason": "not_owner"}

request={"user_id":"u9","role":"admin","object_id":"483"}, ownership={"483":"u2"}
    -> {"allowed": True, "reason": "admin"}

request={"user_id":"u1","role":"user","object_id":"999"}, ownership={"483":"u2"}
    -> {"allowed": False, "reason": "not_found"}
```

### Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

### How to run

```
cd problem && python -m pytest tests.py -q
```

### Starter code:

```
"""Starter code for 5.1.c. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import Dict


def solve(request: Dict[str, str], ownership: Dict[str, str]) -> Dict[str, object]:
    """Decide whether an API request is authorized (BOLA check).

    Args:
        request: {"user_id": str, "role": str, "object_id": str}
        ownership: dict mapping object_id -> owner user_id
    Returns:
        {"allowed": bool, "reason": str} where reason is one of
        "owner", "admin", "not_owner", "not_found".
    """
    # TODO: look up request["object_id"] in ownership.
    #   - not present -> {"allowed": False, "reason": "not_found"}
    #   - owner matches request["user_id"] -> {"allowed": True, "reason": "owner"}
    #   - else if request["role"] == "admin" -> {"allowed": True, "reason": "admin"}
    #   - else -> {"allowed": False, "reason": "not_owner"}
    raise NotImplementedError("implement solve() — see prompt.md")
```

## 5.1.d · Data breaches and their impact

*3 pts · auto-graded on the site*

### Coding problem 5.1.d: Score data breach severity

**Skill:** Analyze Risk · **Maps to:** 5.1.C · **Language:** Python 3 (pytest)

### Problem

Build the severity scorer a security team's intake triage tool would run on each incoming breach report. Given the categories of data exposed, the number of records affected, and how many days it took to detect the breach, compute a total severity score and a label.

### Scoring rubric

Start at `score = 0`.

**Data category points** (sum over all categories present in the report, each counted once):

- `"pii"` (personally identifiable information): +40
- `"financial"`: +40
- `"health"`: +40
- `"auth_plaintext"` (plaintext passwords/secrets): +40
- `"auth_hashed"` (hashed/salted passwords): +15
- `"email"`: +10
- `"public"`: +0

**Scale points** (based on `record_count`):

- `< 100`: +0
- `100-9,999`: +10
- `10,000-999,999`: +20
- `>= 1,000,000`: +30

**Detection-speed points** (based on `days_to_detect`):

- `<= 1`: +0
- `2-30`: +10
- `> 30`: +20

### Label thresholds (applied to the total score)

- `score >= 80` -> `"critical"`
- `score >= 50` -> `"high"`
- `score >= 20` -> `"medium"`
- else -> `"low"`

### Function contract

Implement `solve(report: dict) -> dict` in `starter.py` (see `interface.md` / `blackbox.yaml`).

### Input

`report`: `{"categories": list[str], "record_count": int, "days_to_detect": int}`

### Output

`{"score": int, "label": str}`

### Examples

```
{"categories": ["public"], "record_count": 50, "days_to_detect": 1}
    -> {"score": 0, "label": "low"}

{"categories": ["pii", "auth_plaintext"], "record_count": 500000, "days_to_detect": 45}
    -> {"score": 40 + 40 + 20 + 20, "label": "critical"}   # = 120

{"categories": ["email"], "record_count": 200, "days_to_detect": 5}
    -> {"score": 10 + 10 + 10, "label": "medium"}          # = 30
```

### Constraints

- Deterministic, offline, standard library only.
- Multiple categories are additive; unknown category strings contribute 0 points.
- Must pass all cases in `tests.py` (visible + hidden).

### How to run

```
cd problem && python -m pytest tests.py -q
```

**Starter code:**

```
"""Starter code for 5.1.d. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import Dict, List


def solve(report: Dict[str, object]) -> Dict[str, object]:
    """Score a data breach report's severity.

    Args:
        report: {"categories": list[str], "record_count": int, "days_to_detect": int}
    Returns:
        {"score": int, "label": str} where label is one of
        "low", "medium", "high", "critical" per the rubric in prompt.md.
    """
    # TODO: sum data-category points + scale points + detection-speed points,
    #       then map the total to a label per the thresholds in prompt.md.
    raise NotImplementedError("implement solve() — see prompt.md")
```